ROBOTICS SP 2025
UNIVERSITÀ DELLA SVIZZERA ITALIANA

Tower Dropper

Group Z

Paolo Deidda (paolo.deidda@usi.ch)
Francesco Costa (francesco.costa@usi.ch)
Raffaele Perri (raffaele.perri@usi.ch)

https://github.com/USI-Projects-Collection/FinalProject-Robotics.git

Contents

Setup

To run the code follow the setup and execution instructions provided in the `README.md` file included in this repository.

1 Introduction

1.1 Model and Scene

The project use a RobomasterS1 robot model, with the addition of four proximity sensors, placed respectively (Figure 1):

- **Range_0 (Center-right)**: Primary sensor for orbit distance maintenance during left-side orbital maneuvers. Positioned at the robot's right quadrant with a detection range up to 10 meters.
- **Range_1 (Front-right)**: Primary sensor for initial tower detection and forward-right obstacle detection. Critical for the search phase and collision avoidance.
- **Range_2 (Center-left)**: Secondary sensor utilized during obstacle avoidance protocols, enabling the robot to maintain safe distances from obstacles while navigating around them.
- **Range_3 (Front-left)**: Forward collision prevention sensor that triggers emergency avoidance behaviors when obstacles are detected within critical thresholds.

Figure 1: RobomasterS1 robot model with camera and proximity sensors.

The robot is equipped with a camera (`/rm0/camera/image_color`) used for visual processing tasks, such as weak-spot detection and tower identification.
The scene is constructed with a 3D model of the RobomasterS1 robot placed in the center of a flat terrain. The environment includes four towers, each colored in red to indicate that they are active and can be targeted. The towers are positioned at the corners of the scene (ADD SOME OTHER DETAILS ABOUT THE SCENE). **ADD A PICTURE OF THE SCENE HERE.**

2 Workflow of the Project

2.1 Global Path Planning

2.2 Tower Detection and Interaction System

2.2.1 System Overview

2.2.2 System Overview

The tower detection and interaction system represents a sophisticated autonomous targeting solution. The system integrates multiple sensor modalities including range sensors, RGB camera, and odometry to enable a mobile robot to autonomously locate, approach, orbit, and engage tower targets in a dynamic environment. The core architecture follows a finite state machine approach with six primary operational states: *find_tower*, *position_for_orbit*, *orbit_tower*, *avoid_obstacle*, *align_tower*, and *shoot_tower*. This state-based design ensures robust behavior transitions and clear operational logic flow through well-defined transition criteria and sensor-based decision making. The main control loop executes at 60 Hz, providing real-time responsiveness while maintaining computational efficiency.
Key ROS2 topics include velocity commands (`cmd_vel`), odometry data (`odom`), individual range sensor readings (`/rm0/range_0-3`), and camera imagery (`/rm0/camera/image_color`).

2.2.3 Sensor Integration and Data Processing

Range Sensor Configuration and Calibration

The system utilizes four range sensors positioned strategically around the robot platform to provide environmental awareness (as shown in Figure 1).
Each sensor operates through dedicated ROS2 subscribers that continuously update distance measurements at the sensor's native frequency. The system implements sensor fusion by maintaining a real-time comparison between multiple sensor readings to determine the most reliable distance measurements and detect sensor failures or environmental interference. Readings exceeding 10 meters are considered invalid and replaced with the maximum range value. The system also implements a smoothing algorithm to reduce measurement noise:

$$d_{filtered}(t) = \alpha \cdot d_{raw}(t) + (1 - \alpha) \cdot d_{filtered}(t-1) \quad (1)$$

where $\alpha = 0.8$ provides a balance between responsiveness and stability.

Computer Vision System Architecture

The vision system employs a comprehensive computer vision pipeline optimized for real-time tower detection and tracking. The process begins with image acquisition from the robot's RGB camera, followed by color space conversion and multi-stage filtering.
Color Space Processing: The system converts incoming BGR images to HSV color space to achieve robust color-based detection under varying lighting conditions. HSV provides superior color constancy compared to RGB, particularly important for outdoor or variable lighting scenarios.

Multi-Range Color Filtering: To account for the bimodal distribution of red pixels in HSV space, the system implements dual-range thresholding:

$$\text{Mask}_1 = \text{inRange}(\text{HSV}, [0, 100, 100], [10, 255, 255]) \tag{2}$$

$$\text{Mask}_2 = \text{inRange}(\text{HSV}, [160, 100, 100], [179, 255, 255]) \tag{3}$$

$$\text{Mask}_{final} = \text{Mask}_1 \cup \text{Mask}_2 \tag{4}$$

This approach captures both low-end ($0 - 10°$) and high-end ($160 - 179°$) red hues while maintaining robust detection across different illumination conditions.

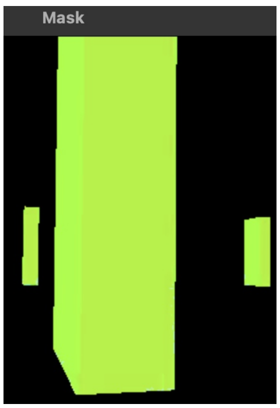

Figure 2: Multi-range color filtering process for red tower detection.

Tower Position Calculation: The system calculates tower position using spatial moments of the binary mask:

$$M_{pq} = \sum_{x,y} x^p y^q \cdot \text{Mask}(x, y) \quad C_x = \frac{M_{10}}{M_{00}} \tag{5}$$

$$p_{tower} = \frac{C_x - W/2}{W/2} \tag{6}$$

where M_{pq} represents the (p, q)-th moment, C_x is the centroid x-coordinate, W is the image width, and $p_{tower} \in [-1, 1]$ represents the normalized tower position.
Tower Visibility Assessment: The system implements a visibility metric based on the ratio of red pixels to total image pixels:

$$R_{red} = \frac{\sum_{x,y} \text{Mask}_{final}(x, y)}{W \times H} \tag{7}$$

Tower visibility is confirmed when $R_{red} > \tau_{visibility}$, where the threshold $\tau_{visibility}$ is dynamically adjusted based on environmental conditions and historical detection performance.

2.2.4 State Machine Implementation and Control Logic

State Transition Architecture

The finite state machine implementation employs a hierarchical structure with clearly defined transition conditions and state-specific behaviors (Figure 3). Each state maintains internal variables for tracking progress and implementing timeouts to prevent infinite loops or deadlock conditions. State transitions are triggered by combinations of sensor readings, timing constraints, and task completion flags. The system implements hysteresis in transition conditions to prevent oscillation between states due to sensor noise or borderline threshold conditions.

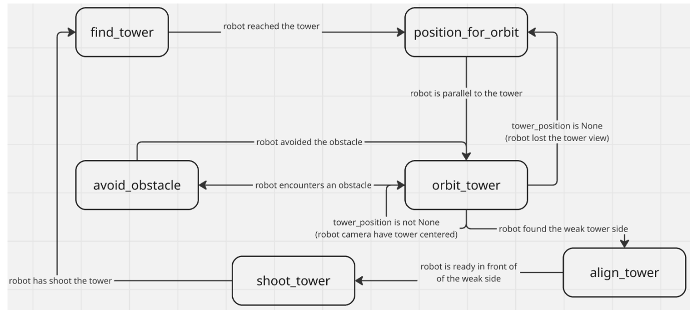

Figure 3: Robot state machine diagram illustrating the tower detection and interaction workflow.

Tower Discovery Phase: *find_tower* State

The robot executes counterclockwise rotation at $\omega_{search} = 0.5$ rad/s while continuously monitoring the front-right range sensor (Range_1). This angular velocity provides optimal balance between search speed and sensor update rates, ensuring no potential targets are missed during rotation. **Detection Criteria:** Tower detection occurs when the front-right sensor measurement satisfies:

$$d_{Range1} \leq d_{target} + \delta_{tolerance} \tag{8}$$

where $d_{target} = 2.0$ meters represents the desired orbital radius and $\delta_{tolerance} = 0.1$ meters provides measurement uncertainty accommodation.
Multi-Sensor Validation: Upon initial detection, the system performs cross-validation using adjacent sensors to confirm target authenticity and eliminate false positives from environmental artifacts or sensor noise. The validation process requires consistent readings across multiple sensor cycles to prevent premature state transitions.

Orbital Positioning Phase: *position_for_orbit* State

This critical state ensures proper spatial relationship between robot and tower before initiating orbital maneuvers.
Geometric Positioning Strategy: The system executes a controlled rotation to position the detected tower relative to the robot's right side, enabling subsequent left-side orbital motion. The positioning maneuver uses angular velocity $\omega_{position} = 0.5$ rad/s while monitoring the back-right sensor (Range_0).
Position Validation Criteria: Successful positioning requires the back-right sensor to detect the tower within acceptable bounds:

$$d_{Range0} \leq d_{target} + \delta_{position} \tag{9}$$

where $\delta_{position} = 0.5$ meters provides sufficient tolerance for initial orbital entry while maintaining proximity to the target distance.

Timeout and Recovery Mechanisms: The state implements timeout protection to prevent infinite positioning attempts. If positioning cannot be achieved within a predetermined time window, the system reverts to the search state with modified search parameters.

Orbital Navigation Control: *orbit_tower* State

The orbital control system represents the most sophisticated aspect of the navigation architecture, combining multiple control loops for stable circular motion around the detected tower.

Multi-Loop Control Architecture: The orbital controller implements two primary control loops operating in parallel:

1. **Distance Regulation Loop**: Maintains constant radial distance using range sensor feedback
2. **Angular Tracking Loop**: Centers tower in camera field of view using vision feedback

Distance Control Implementation: The distance regulation employs PID control with anti-windup protection:

$$e_d(t) = d_{measured} - d_{target} \tag{10}$$

$$u_d(t) = K_p e_d(t) + K_i \int_{t_0}^{t} e_d(\tau) d\tau + K_d \frac{de_d(t)}{dt} \tag{11}$$

The integral term includes saturation limits to prevent windup:

$$\int_{t_0}^{t} e_d(\tau) d\tau = \begin{cases} I_{\max} & \text{if } \int e_d(\tau) d\tau > I_{\max} \\ I_{\min} & \text{if } \int e_d(\tau) d\tau < I_{\min} \\ \int e_d(\tau) d\tau & \text{otherwise} \end{cases} \tag{12}$$

where $I_{\max} = 5.0$ and $I_{\min} = -5.0$ prevent excessive integral buildup.

Velocity Coordination: The orbital motion combines base velocities with correction terms from both control loops:

$$\begin{aligned} v_{linear} &= v_{base} \cdot f_{distance}(e_d) \omega_{angular} \\ &= \omega_{base} + \alpha \cdot u_d + \beta \cdot p_{tower} \end{aligned} \tag{13}$$

where:

$$f_{distance}(e_d) = \begin{cases} 0.8 & \text{if } e_d > 0.2 \text{ (too far)} \\ 1.2 & \text{if } e_d < -0.2 \text{ (too close)} \\ 1.0 & \text{otherwise} \end{cases} \tag{14}$$

The velocity adaptation function $f_{distance}$ provides reactive speed adjustments based on distance error magnitude, enhancing convergence to the target orbital radius. The system implements comprehensive velocity limiting to ensure safe operation: $v_{linear} \in [0.05, 0.3]$ m/s $\omega_{angular} \in [-\infty, -0.1]$ rad/s. The angular velocity constraint ensures consistent counterclockwise motion while preventing excessive rotation rates that could destabilize the orbital behavior.

Obstacle Avoidance Protocol: *avoid_obstacle* State

The obstacle avoidance system implements a dual-objective navigation strategy that maintains safe distances from obstacles while preserving awareness of the primary tower target.

Obstacle Detection and Classification: Obstacle detection triggers when front sensors detect objects within the critical threshold.

Secondary Orbital Behavior: Upon obstacle detection, the system initiates a secondary orbital pattern around the detected obstacle using the back-left sensor (Range_2) for guidance.

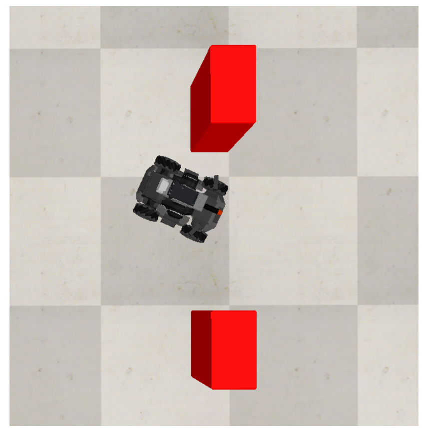

Figure 4: Robot avoiding the top obstacle encountered during orbital navigation.

Return Navigation Logic: The system implements logic for returning to the primary tower after obstacle avoidance. The return condition requires:

$$\begin{aligned} (d_{Range2} > d_{clearance}) \wedge (d_{Range0} < d_{tower,\max}) \\ \wedge \, \text{flag}_{almost\_avoided} \end{aligned} \tag{15}$$

where $d_{clearance} = 10.0$ meters indicates obstacle clearance and $\text{flag}_{almost\_avoided}$ prevents premature return attempts.

2.3 Weak-Spot Detection & Shooting

Tower Alignment and Targeting System

The system requires alignment algorithm to be used in order to detect the weak side of the tower which, when working with rectangular bases, simply equates to the largest side. The problem could easily be solved by coloring different sides of the tower or by including a special mark on the target side. This solution would harm the implementation on the long run, as we would always be forced to explicitely distinguish sides by altering the visual properties of towers, leading to more constrained and less realistic results.

Alignment Detection

The method we developed to detect the weak side of the tower makes use of the binary mask already in use to assess the position of the tower. In this case we take advantage of the discretization limits of pixels. The easiest, and surprisingly reliable, method to detect whether the robot is looking at a flat face of the tower relies in fact on the distribution of colored pixels in the mask, in particular around the base of the tower. If the number of continuous colored pixels in the first row of the mask containing any non-black pixel (starting from the bottom) is around as high as the highest number of continous pixels in a row of the mask, we can confidently say we are looking at a flat face of the tower. Visually this would mean that in the binary mask we see a sharp edge at the bottom of the tower, instead of a 'pixelated' line.

Sensor-Based Geometric Validation

Once the robot is confident of being parallel to the largest side of the tower we follow an iterative process in order to turn the robot until it looks straight towards its target. The alignment process follows a systematic approach:

1. **Initial Rotation**: Controlled rotation at $\omega_{align} = -0.3$ rad/s to scan for geometric variations
2. **Asymmetry Detection**: Continuous monitoring of range sensor differences to identify tower edges
3. **Symmetry Convergence**: Fine positioning adjustments to achieve bilateral sensor equality
4. **Alignment Correction**: Final correction of the alignment following rotation direction

Projectile Launch System

The *shoot_tower* state executes the engagement sequence through integration with the CoppeliaSim physics engine. The system calculates projectile spawn position using coordinate transformation:

$$\mathbf{R} = \begin{bmatrix} \cos(\psi) & -\sin(\psi) & 0 \\ \sin(\psi) & \cos(\psi) & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{16}$$

$$\mathbf{p}_{projectile} = \mathbf{p}_{robot} + \mathbf{R} \cdot \mathbf{t}_{blaster} \tag{17}$$

where ψ represents the robot's roll angle and $\mathbf{t}_{blaster} = [0.2, 0, 0.2]^T$ meters defines the blaster offset from the robot center. The projectile receives initial velocity $\mathbf{v}_{projectile} = \mathbf{R} \cdot [3.0, 0.0, 3.0]^T$ m/s, providing forward momentum with upward trajectory compensation for ballistic flight.

3 System Integration

3.1 ROS 2 Node Graph & Communication Channels

3.2 Integration Strategy and Version Control

3.3 Observed Failure Modes & Mitigations

4 Results

5 Conclusion & Future Improvements

5.1 Potential Improvements

5.2 Unexplored Ideas